Single-Node Private Cloud Deployment Using OpenStack and Kolla-Ansible

Project Documentation

Abstract

This project involved deploying a single-node private cloud environment using OpenStack Bobcat (2023.2) and Kolla-Ansible 17.1.0 on Ubuntu Server 22.04 running in VirtualBox. Kolla-Ansible was used to automate the deployment of OpenStack services as containers. The environment included compute, networking, identity, dashboard, block-storage and object-storage services. Virtual loopback devices were prepared for Cinder LVM block storage and Swift XFS object storage, with Swift ring files created for object, container and account services. Nextcloud was integrated with Swift using Keystone v3 authentication. A Python-based synchronization script using python-swiftclient and keystoneauth1 was implemented to upload files from a local watch folder to Swift containers. Tailscale VPN provided remote access for testing.

Introduction

A private cloud provides computing, networking and storage resources within an environment managed by its administrator. OpenStack is an open-source cloud platform that brings these resources together through services and APIs. This project explores the deployment of OpenStack on a single virtualized Ubuntu server, providing a practical environment for learning cloud infrastructure administration and service integration.

Kolla-Ansible simplifies the deployment process by using Ansible playbooks and container images to configure and run OpenStack services. The project also extends the basic cloud environment by integrating Swift object storage with Nextcloud and enabling remote access through a VPN.

Motivation and Objectives

The objective was to build and operate a functional private cloud in a resource-constrained, single-node lab environment and gain practical experience with OpenStack deployment, storage backends, identity integration and remote access.

- Deploy OpenStack Bobcat using Kolla-Ansible on Ubuntu Server 22.04.
- Configure core OpenStack services for compute, networking, identity and dashboard access.
- Prepare block and object storage using Cinder LVM and Swift XFS.
- Create Swift ring files for object, container and account services.
- Integrate Nextcloud with Swift using Keystone v3 authentication.
- Automate file uploads to Swift using a Python synchronization script.
- Enable remote connectivity using Tailscale VPN.

Tools and Technologies Used

Ubuntu Server 22.04: Operating system hosting the single-node OpenStack deployment.

VirtualBox: Virtualization platform used to run the Ubuntu server environment.

OpenStack Bobcat 2023.2: Private cloud platform providing infrastructure services.

Kolla-Ansible 17.1.0: Deployment and configuration automation for OpenStack services.

Ansible: Executes deployment playbooks and configuration tasks.

Docker: Runs the containerized OpenStack services deployed through Kolla-Ansible.

Nova: Compute service used to manage virtual machine instances.

Neutron: Networking service used to provide virtual networking.

Keystone: Identity service providing authentication and service access.

Horizon: Web dashboard for interacting with OpenStack resources.

Cinder: Block storage service configured with an LVM backend.

Swift: Object storage service configured with XFS-backed storage and ring files.

Nextcloud: File-management interface integrated with Swift object storage.

Python, python-swiftclient and keystoneauth1: Used to authenticate and automate file uploads to Swift.

Tailscale: VPN-based remote connectivity for authorized access and testing.

System Architecture

The deployment runs on a single Ubuntu Server 22.04 virtual machine hosted in VirtualBox. Kolla-Ansible automates the deployment of OpenStack services, which run as Docker containers. Nova provides compute management, Neutron provides virtual networking, Keystone handles identity and authentication, and Horizon provides a web-based management interface. Cinder and Swift provide block and object storage respectively.

For storage, virtual loopback devices were prepared for the Cinder LVM and Swift XFS backends. Swift uses ring files to map object, container and account services across the configured storage resources. Nextcloud connects to Swift through Keystone v3 authentication, while a Python script uploads files from a local watch folder into Swift containers. Tailscale provides remote connectivity to the environment.

Implementation Details

1. Preparing the Virtualized Environment

A virtual machine was created in VirtualBox and configured with Ubuntu Server 22.04. The operating system was prepared with the dependencies and network configuration required for the OpenStack deployment.

2. Installing and Configuring Kolla-Ansible

Kolla-Ansible 17.1.0 was installed and used to automate the deployment of OpenStack Bobcat 2023.2. Ansible inventory and Kolla configuration files were prepared for the single-node environment. The deployment workflow was used to bootstrap the host, validate the configuration and deploy the OpenStack services as Docker containers.

3. Deploying OpenStack Services

The single-node environment was configured with Nova, Neutron, Keystone, Horizon, Cinder and Swift. These services provide the compute, networking, identity, dashboard, block-storage and object-storage capabilities required for the private cloud environment.

4. Configuring Cinder Block Storage

Virtual loopback storage was prepared for the Cinder LVM backend. This allowed the single-node lab environment to provide block-storage resources without requiring a separate physical storage device. Cinder was configured as the OpenStack block-storage service for volume management.

5. Configuring Swift Object Storage

Swift storage was prepared using a virtual storage device formatted with XFS. Ring files were manually created for the object, container and account services. These rings provide the mapping information Swift uses to manage object and metadata placement in the configured storage environment.

6. Integrating Nextcloud with Swift

Nextcloud was integrated as a file-management interface on top of the OpenStack object-storage service. Keystone v3 authentication was used to authenticate access to Swift. This connected the user-facing file-management layer with the object-storage backend provided by OpenStack.

7. Automating File Uploads with Python

A Python-based hybrid cloud synchronization script was implemented using python-swiftclient and keystoneauth1. The script was designed to monitor a local watch folder and upload files to the configured Swift containers after authenticating with the OpenStack identity service. This reduced the need for manual file uploads and demonstrated programmatic interaction with OpenStack object storage.

8. Configuring Remote Access

Tailscale VPN was configured to provide remote connectivity to the private cloud environment. Remote access was tested from an external network, including file-upload operations from mobile and WAN-connected devices.

Challenges Encountered and Resolutions

The deployment required troubleshooting across the automation, container and networking layers. The issues recorded during implementation included:

- Ansible version and dependency mismatches during Kolla-Ansible setup.
- Missing ansible.netcommon collection required by the automation environment.
- Docker SDK and Requests library incompatibility, addressed by pinning Requests to version 2.31.0.
- HAProxy startup timeout associated with a missing base IP address on the network interface.

Resolving these issues required checking deployment dependencies, validating host network configuration and adjusting the software environment before continuing with the deployment.

Results and Outcomes

- Deployed a single-node OpenStack Bobcat 2023.2 environment using Kolla-Ansible 17.1.0.
- Configured core compute, networking, identity and dashboard services.
- Prepared Cinder LVM block storage and Swift XFS object storage using virtual loopback devices.
- Created Swift ring files for object, container and account services.
- Integrated Nextcloud with Swift using Keystone v3 authentication.
- Implemented Python-based file uploads to Swift using python-swiftclient and keystoneauth1.
- Configured Tailscale VPN and tested remote access and file uploads.

Learning Outcomes

- OpenStack service architecture and Kolla-Ansible deployment workflows.
- Containerized cloud service deployment using Docker.
- Block and object storage configuration using Cinder and Swift.
- Swift ring architecture and object-storage concepts.

- Keystone v3 authentication and application integration.
- Python-based automation using OpenStack client libraries.
- Linux administration, dependency troubleshooting and VPN connectivity.

Conclusion

The project demonstrated the deployment of a single-node private cloud using OpenStack Bobcat and Kolla-Ansible on Ubuntu Server 22.04. The environment combined compute, networking, identity, dashboard and storage services. Cinder and Swift were configured using virtual storage devices, and Nextcloud was integrated with Swift through Keystone v3 authentication. A Python synchronization script automated file uploads to Swift, while Tailscale provided remote connectivity. The implementation provided practical experience in private cloud deployment, storage configuration, service integration, automation and troubleshooting.

Scope and Deployment Note

This report describes the single-node virtualized project only. It is separate from the later three-node bare-metal OpenStack deployment and the Kubernetes cluster deployed within the multi-node environment. Storage devices in this single-node lab were virtual loopback devices; this report does not claim physical storage redundancy or high availability.